

COS 135 Individual Assignment 2

Due: Monday 02/09/26 End of the day

This assignment has 3 sections (part #1, #2, and #3) in 4 pages.

Warning: To avoid compilation errors, do not copy-paste code from PDFs, Word documents, or websites. These sources often contain invisible Unicode characters (such as zero-width spaces) or incorrect formatting symbols (such as “ instead of ") that will cause the C compiler to fail.

Part #1 (30pts): In your Ubuntu terminal, perform the following steps and capture one or two screenshots showing the full workflow (terminal output must be visible, username included).

1. Open a terminal and check your current working directory (i.e., folder) using **pwd** command
2. List all the files in your current directory using **ls** command
3. Create a new directory using **mkdir my_folder** command (note: *mkdir* is the make directory command, *my_folder* is the directory name. Notice the space character after the command)
4. Change your current directory to the new directory created using the **cd my_folder** command (notice the space character between *cd* and *my_folder*)
5. Verify the new current directory again using the **pwd** command
6. Create two empty files using the **touch info.txt my_code.c** command (notice the space characters after the touch command and the first filename)
7. Check more information about the new files using **ls -l info.txt my_code.c** command (note: the output provides more information about the file permissions, owner's username, size, and date and time created)
8. Delete one file using **rm info.txt** command
9. Change your current directory to the previous (original) directory using **cd ..** command (notice the space character between *cd* and *..*)
10. Now delete the folder you created along with the remaining file using **rm -rf my_folder** command (command structure: *rm <one space> -rf <one space> my_folder*). Here *-rf* stands for recursively and forcefully. *rm* command will not work on a folder directly; you need to at least use *-r* option to remove all the files inside the directory before removing the directory itself. When you use the *-f*, it will remove the files forcefully - even a file is currently open.

Note: It is not a good practice to use space characters when defining file names and directory names (for instance, instead of *my file*, use *my_file*). Modern operating systems (e.g., Windows, Mac OS, Ubuntu, Fedora) can handle the space characters in files and folder names. However, some Unix and older versions (especially servers and small device operating systems such as FreeRTOS and Embedded Linux) may not support the space character in file or folder names. Furthermore, in Unix terminal if you issue the command **mkdir my file**, it will create two new folders named *my* and *file*.

Part #2 (20pts): Fill in the blanks (submit your completed source code/s without errors).

(a) Use suitable format specifiers to complete the following code
(submit the completed source code in a .c file).

Note: The output should show as **Character having ASCII value 88 is X.**

```
#include <stdio.h>

int main()
{
    char chr = 88;
    printf("Character having ASCII value ____ is ____.\n", chr, chr);
    return 0;
}
```

(b) Complete the printf() function with suitable format specifiers to output the data in variables
(submit the completed code in a .c file).

```
#include <stdio.h>

int main() {
    int i_number = 12;
    float f_number = 3.125;
    double d_number = 7.5345345;
    char character = 'T';
    char text[] = "COS 135";

    printf("_____", _____);
}
```

Part #3 (50pts): Submit your source code without errors together with the screenshot as explained below.

Write a C program that produces a formatted “system report” using `printf()` and basic C concepts.

File name: `system_report.c`

Program Requirements

When compiled and run, your program must output the following information in a clean, readable format:

1. A title header (your choice of formatting)
2. Your name
3. Course number (COS 135)
4. Current semester
5. The number of command-line arguments passed to the program
6. A formatted table containing system-related values (hardcoded)

Example output structure (exact formatting may differ):

```
-----  
System Report  
-----  
Name: Jane Doe  
Course: COS 135  
Semester: Spring 2026  
Arguments passed: 1  
  
Component          Value  
-----  
CPU Cores           8  
Memory (GB)         16  
Disk Size (GB)      512
```

Technical Requirements:

- Use at least three different format specifiers
- Use field width and alignment
- Use variables to store component values (CPU, memory, and disk size) before printing
- Compile using:
 - `gcc system_report.c -o system_report`
- Run using:
 - `./system_report`

(5pts) Required Explanation (Important)

Add comments in your code answering the following:

1. Explain what one `printf()` line in your program does
(choose the line you think is most important).
2. Explain why field width or alignment was used in your table output.

These explanations must refer to your own code, not generic definitions.

(5pts) Reflection (2–3 sentences, required)

Add answer as a comment at the end of your source code.

What part of this program did not work the first time you tried it, and how did you fix it?

Submission Requirements

Submit:

`system_report.c`

One screenshot showing:

successful compilation

program execution and output

Grading Breakdown

- Correct compilation and execution: 20 pts
- Output formatting and alignment: 20 pts
- Code clarity and required comments: 5 pts
- Reflection: 5 pts